COMP 3920 Database Systems 1

MS SQL
Party Database

Introduction:

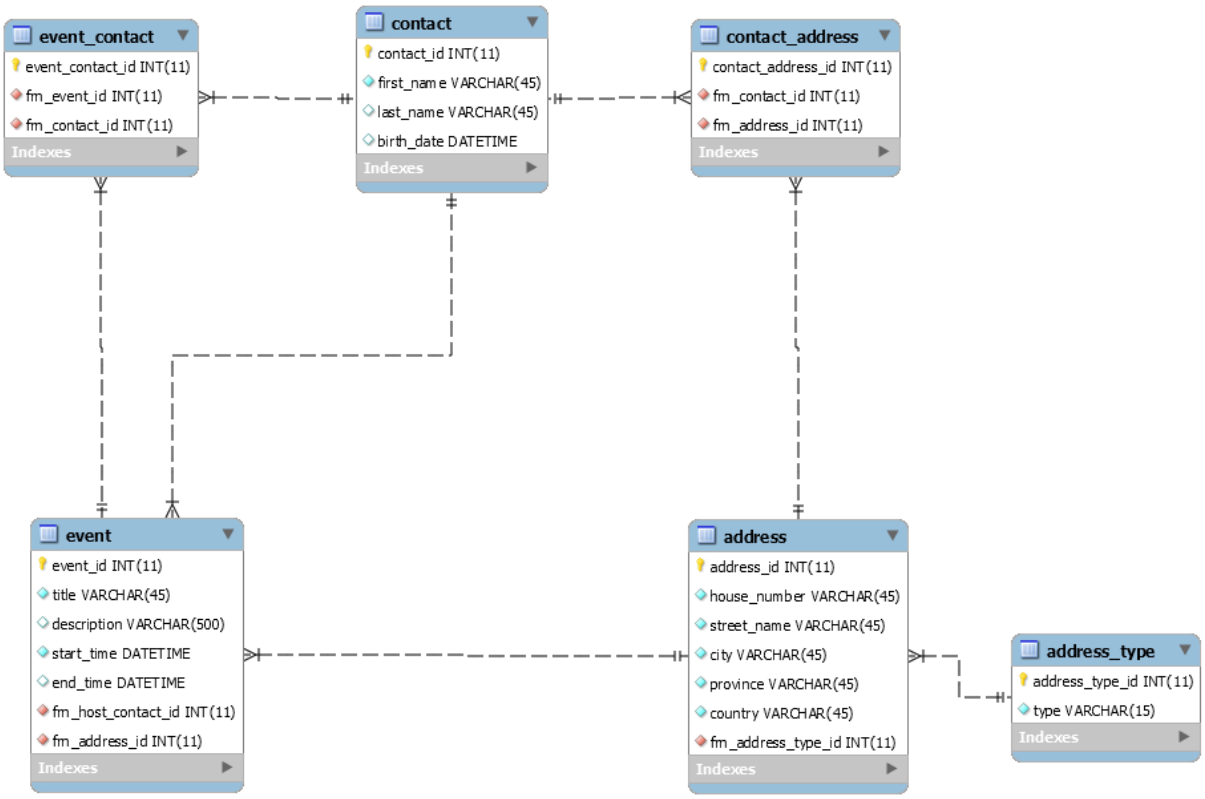
The goal of this document is to serve as a walkthrough creating a simple database with tables, constraints, foreign keys and various data types in MS SQL Server.

For this document, we will be using command prompt commands and Microsoft's SQL Server Management Studio (SSMS).

We will be creating 6 tables:

1. contact (i.e. people)
2. address (i.e. people's houses or party venues)
3. contact_address (i.e. where do people live, or work?)
4. address_type (ex: home, work, public)
5. event (i.e. parties)
6. event_contact (i.e. Invitees - who is coming to the party?)

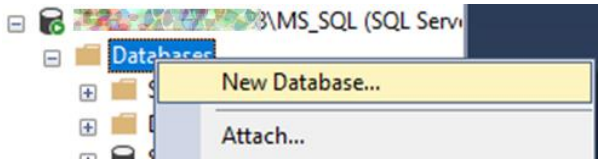
Database Entity Relationship Diagram (ERD) shown below:



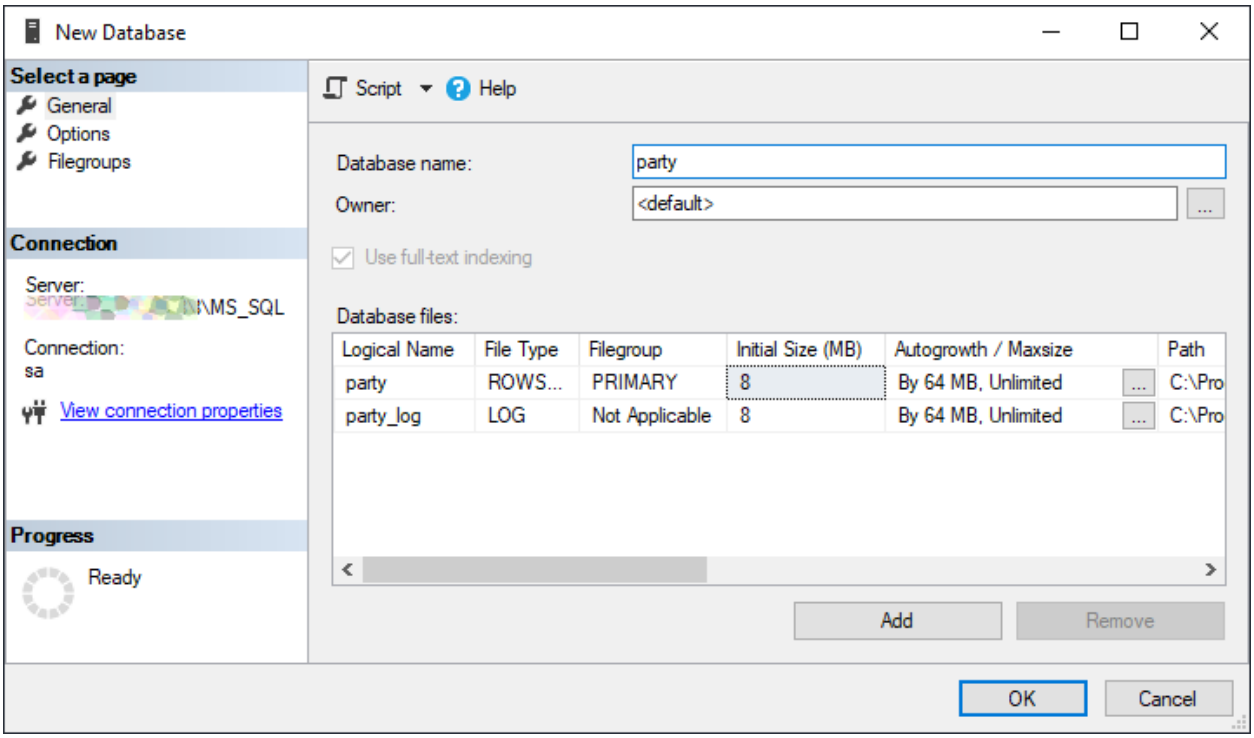
Step 1:
Open SQL Server Management Studio (SSMS).

Step 2:
Create a new database called "party".

Right click on "Databases" in the Object Explorer.

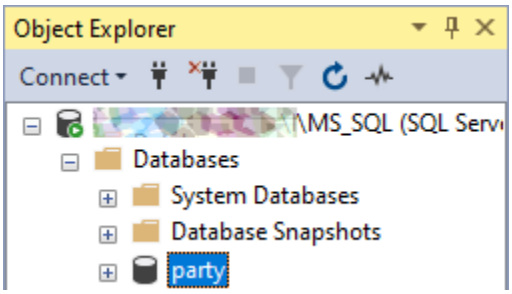


Click on "New Database...".



Type "party" for the database name, click OK.

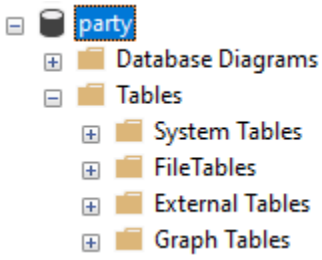
You should see the party database now in the Object Explorer. If you don't, click the refresh button to refresh the database list.



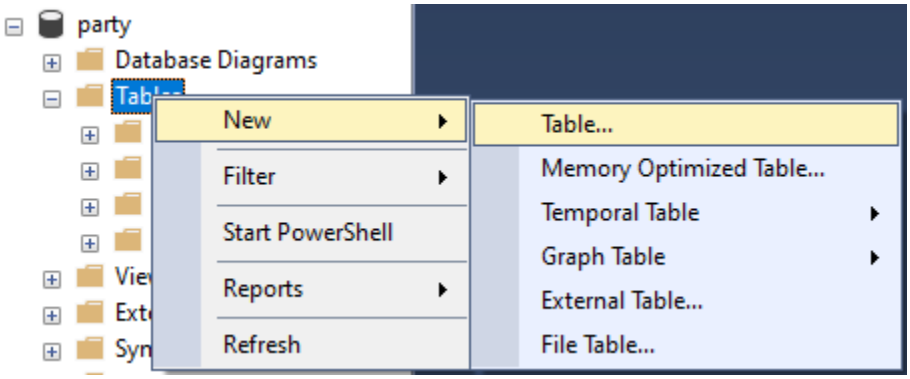
Step 3:
From within the party database, you can create new tables.

Expand the party database by clicking the plus symbol next to the party database in the Object Explorer.

You can also expand the tables by again clicking the plus symbol.



Right click on Tables and click "New" to create a new table.



Creating a table for MS SQL Server in the SSMS interface is similar to working with MySQL and Workbench, but some of the names are different. We'll also need to know how to do things like create a primary key, how to make it auto_increment, and how to create constraints like unique key and foreign keys.

First let's create some columns in our new table.

Our interface within SSMS for creating a new table looks like this:

Column Name	Data Type	Allow Nulls
		☐

We can add columns here – example:

Column Name	Data Type	Allow Nulls
address_id	int	☑
house_number	nvarchar(45)	☐
street_name	nvarchar(45)	☐
city	nvarchar(45)	☐
province	nvarchar(45)	☐
country	nvarchar(45)	☐
frn_address_type_id	int	☐
		☐

Make sure to properly set the data type and uncheck the "Allow Nulls" checkbox if the column is not allowed to be Null.

For the data type, MS SQL Server uses `nvarchar` instead of `varchar` that MySQL uses.

Next let's add a primary key.

To do this, right click on the column you want to be a primary key (usually the first column in the table) and then click on "Set Primary Key".

Column Name	Data Type	Allow Nulls
address_id	int	☑
		☐
		☐
		☐

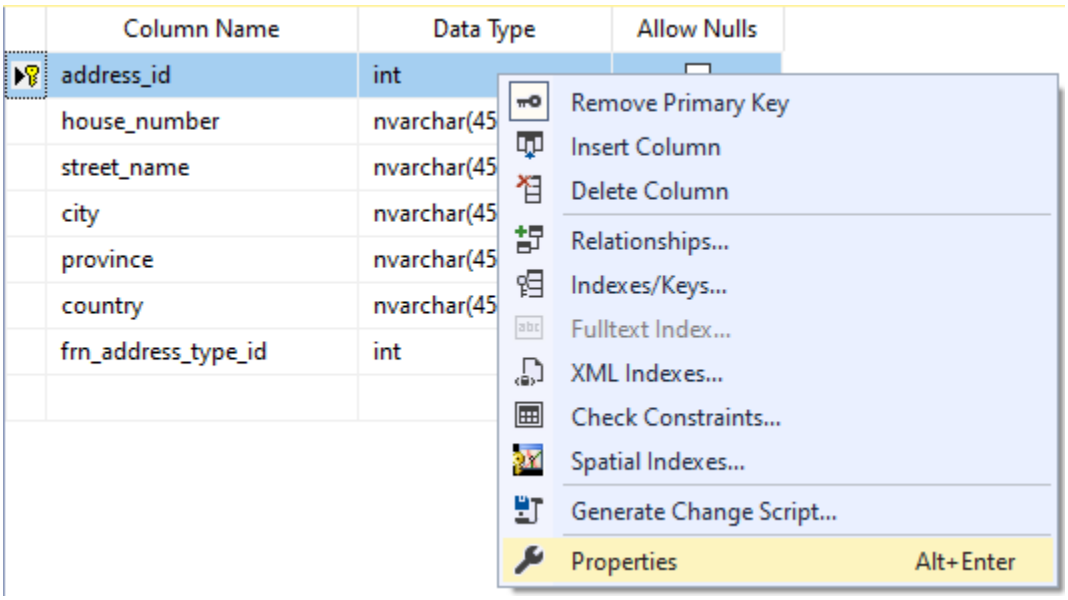
Set Primary Key
Insert Column
Delete Column

Once you've done this, you should see your column has a yellow key next to it and the column has removed the "Allow Nulls" checkmark because primary keys cannot be null.

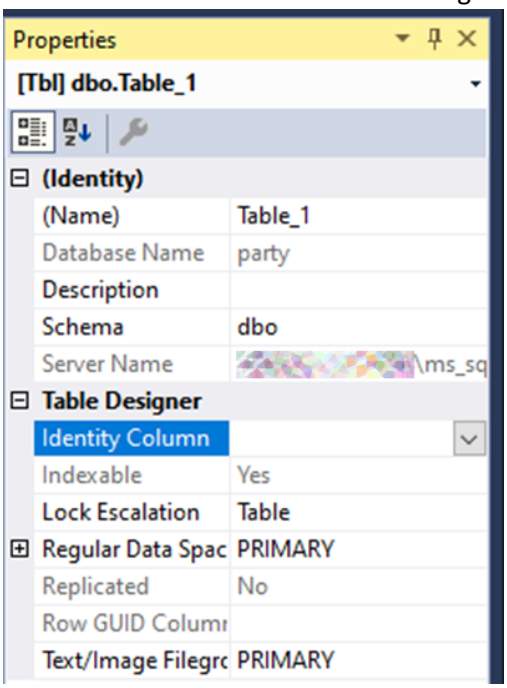
	Column Name	Data Type	Allow Nulls
	address_id	int	☐

We also want to make our primary key an `auto_increment` field, i.e., we want it to automatically choose the next highest number for our primary key when we add new rows. However, auto_increment is a MySQL specific term and keyword. In MS SQL it is call an **Identity** column.

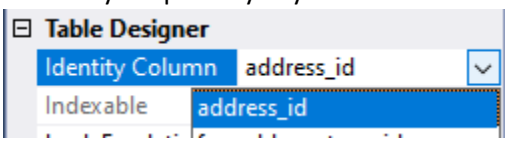
To set an Identity column, open up the table's properties panel by right clicking on one of the columns and click "Properties".



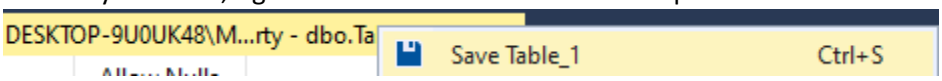
In the properties panel, you should see an "Identity Column" entry with a dropdown where you can pick a column to have an auto incrementing value.



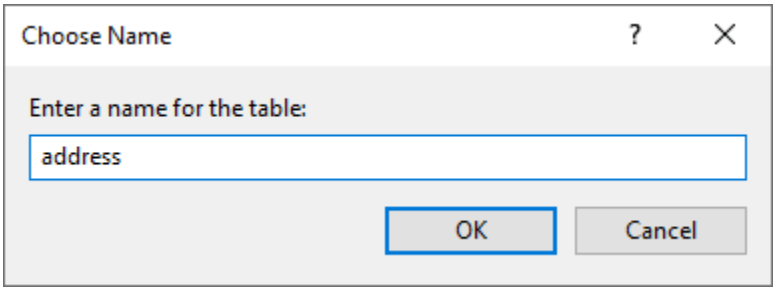
Click on your primary key column to set it as the Identity column.



To save your table, right click on the table's tab at the top of the table definition and click "Save Table1":



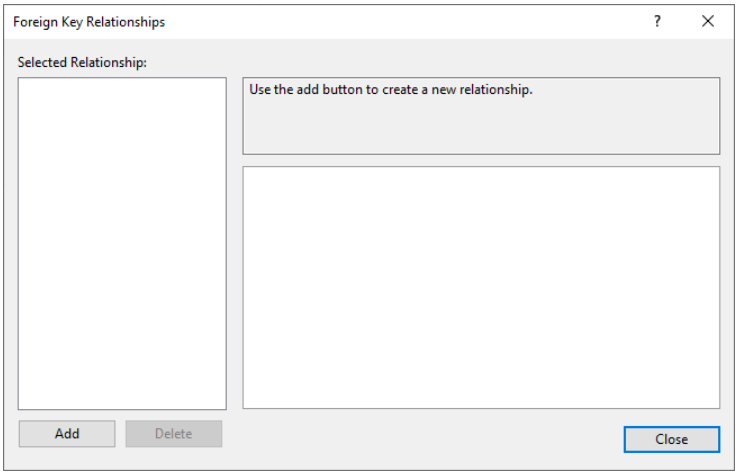
You'll be prompted for the name of your new table. Provide the appropriate name.



You'll also want to establish your foreign keys.

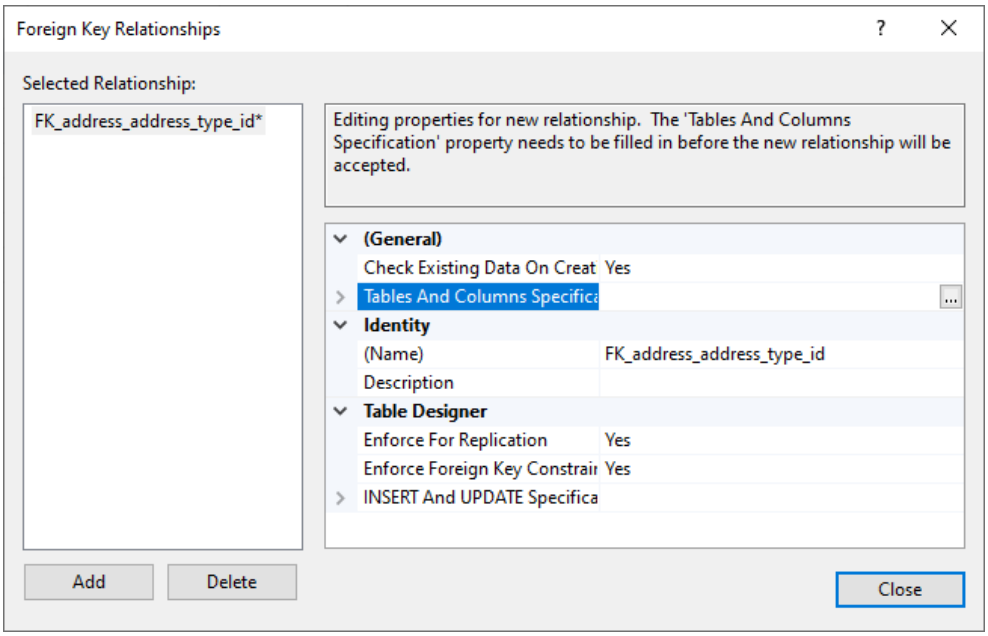
Remember that foreign keys (often abbreviated as FKs) must refer to a column in another table – typically its primary key (PK). IMPORTANT. Because of this, you need to create PKs ***before*** you create the FKs that reference them.

To create an FK, right click on your table, and click "Relationships"



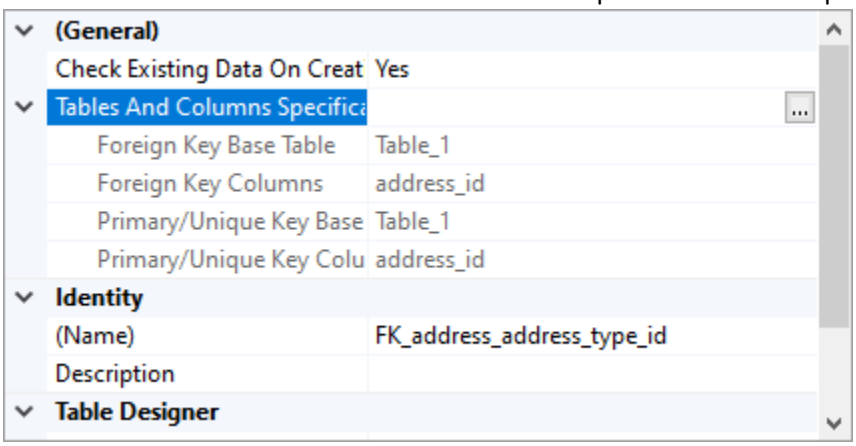
In this Foreign Key Relationships window, click Add.

Provide the FK a name by using this convention:
"**FK**" + **[name of foreign key table]** + "**_**" + **[name of primary key field]**
For example, if the `frn_address_type_id` column in the **`address`** table (FK table) references the **`address_type_id`** column in `address_type` table (PK table), your FK name would be: **`FK_address_address_type_id`**.



Next we need to link the PK and FK tables and columns.

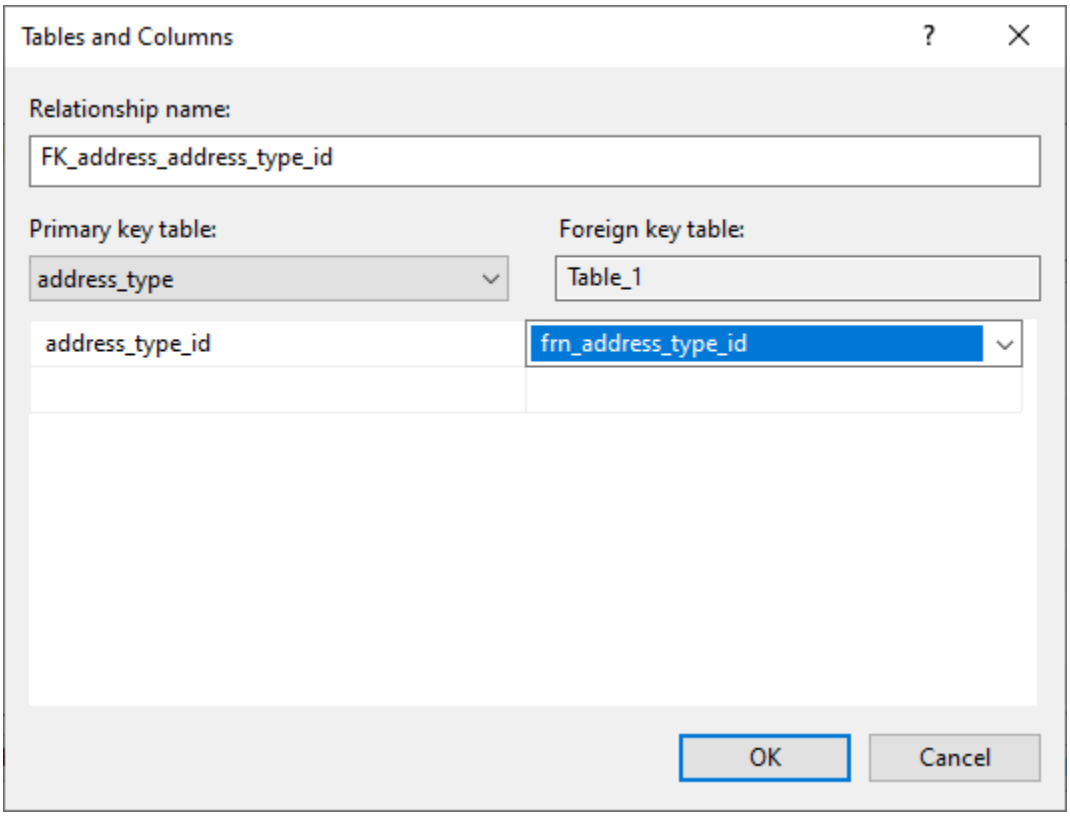
Click the arrow next to the "Tables and Columns Specifications" to expand this section:



In here we need to specify 4 things:

1. The FK table
2. The FK column
3. The PK table
4. The PK column

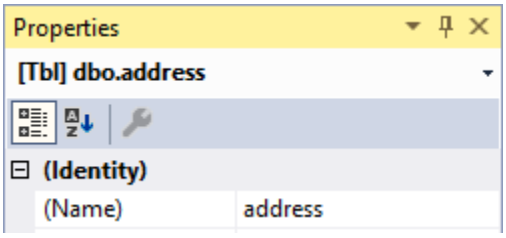
Click on the 3 dots ... to open up the Tables and Columns popup:



Specify the PK table, PK column, and FK column. Note: the FK table will be already chosen and cannot be changed (it's the name of the table you are currently editing).

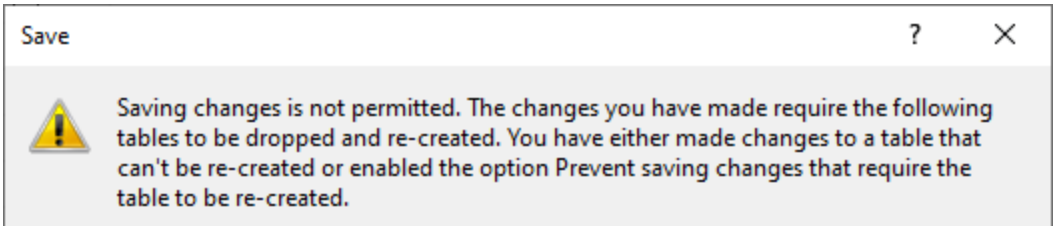
Click OK, then click Close.

Note: the Properties panel is a great place to set the name of your table if you don't want to wait until you save your table to set the table's name.



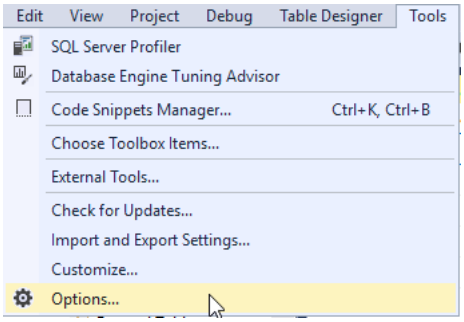
Remember to save your tables .

If you need to change a table after you've already created it, you might get an error something like this:



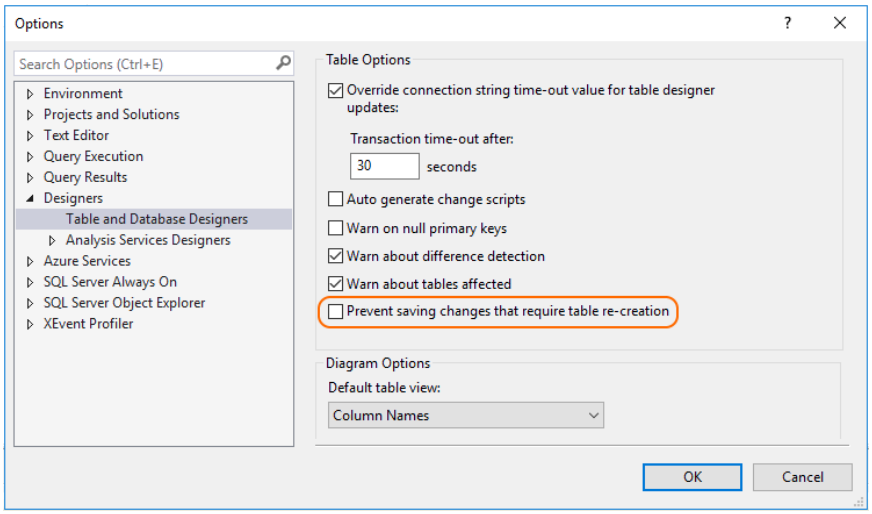
Sometimes Microsoft SQL Server Management Studio gives us errors when we try to edit the definition of a table. There is a good reason for this too! If we change the definition of a table we can lose data from our table. Imagine this scenario, if we accidentally delete the birth_date column on a large table of employees, when we save the table definition, all of those birth dates stored in that column disappear! Even if we add back that birth_date column later, we can't recover the data (unless we have a backup to recover from). So we need to be ***absolutely*** sure we want to make changes to the table definition before saving. Since we are only playing with test data in these labs, we can safely disable this feature (and allow changing a table's definition).

To allow changing a table's definition after it was created, go to "Tools", then "Options".



Find "Designers" and "Table and Database Designers" in the tree view on the left.

Find "Prevent saving changes that require table re-creation" on the right and uncheck it.



With this checkbox unchecked, we can now alter our tables with getting this error message. But be careful, it may also allow us to destroy data in our tables too!

Create the following tables:

address_type

Field	Type	Null	Key	Extra
address_type_id	int	NO	PRI	Identity Column
type	nvarchar(15)	NO		

address

Field	Type	Null	Key	Extra
address_id	int	NO	PRI	Identity Column
house_number	nvarchar(45)	NO		
street_name	nvarchar(45)	NO		
city	nvarchar(45)	NO		
province	nvarchar(45)	NO		
country	nvarchar(45)	NO		
frn_address_type_id	int	NO		

contact_address

Field	Type	Null	Key	Extra
contact_address_id	int	NO	PRI	Identity Column
frn_contact_id	int	NO		
frn_address_id	int	NO		

contact

Field	Type	Null	Key	Extra
contact_id	int	NO	PRI	Identity Column
first_name	nvarchar(45)	NO		
last_name	nvarchar(45)	YES		
birth_date	datetime	YES		

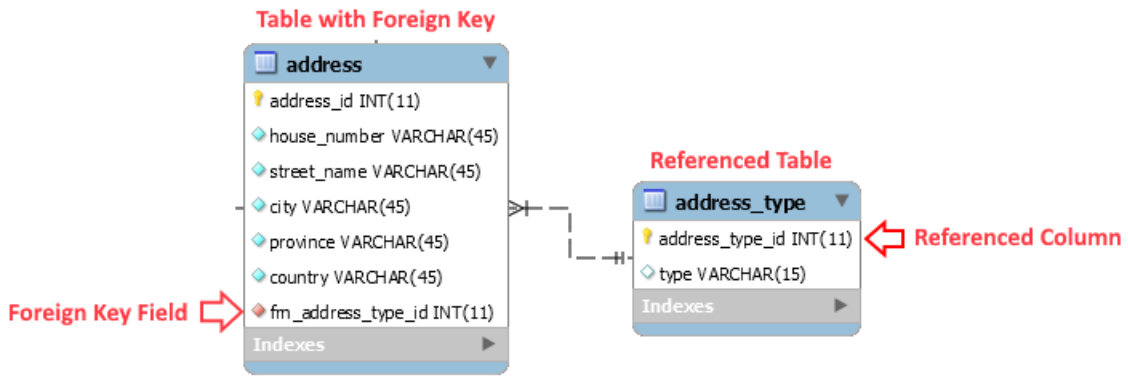
event_contact

Field	Type	Null	Key	Extra
event_contact_id	int	NO	PRI	Identity Column
frn_event_id	int	NO		
frn_contact_id	int	NO		

event

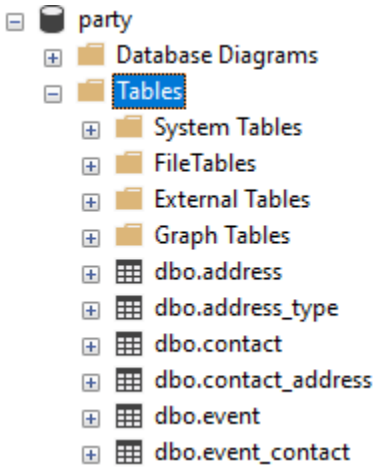
Field	Type	Null	Key	Extra
event_id	int	NO	PRI	Identity Column
title	nvarchar(45)	NO		
description	nvarchar(500)	YES		
start_time	datetime	NO		
end_time	datetime	YES		
frn_host_contact_id	int	NO		
frn_address_id	int	NO		

Step 4:
Before saving each table we will create the foreign keys as shown in the Database ERD.
Let's go through and example of creating the foreign key between `address` and `address_type`.



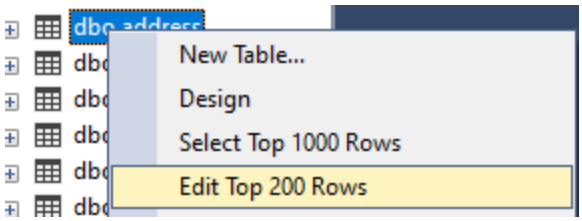
The `address` table has a foreign key called `frn_address_type_id`. We will be prefixing all foreign key fields with `frn`.

Verify the tables have been created in the 'Schema View':
Click Refresh if needed. Ensure all the tables have been added as expected.



Step 5:
Insert data into the tables. We are going to create some data to populate these tables (data shown below).

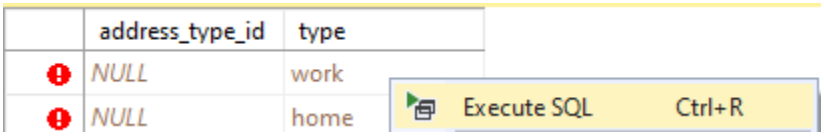
You can enter the data into SSMS's editor by going into the "Edit Top 200 rows" option for the table.



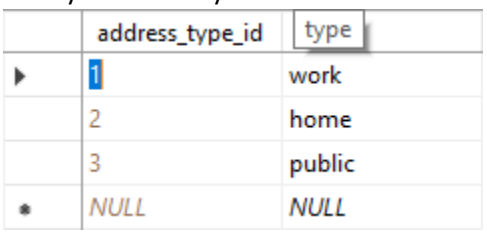
For Identity columns, leave the field blank.

	address_type_id	type
!	NULL	work
!	NULL	home
!	NULL	public
▸*	NULL	NULL

If your Identity field comes back with a red exclamation point , this is ok, to refresh and see the value assigned to the primary key, right click on the table then click "Execute Query".



Now you can see your PK values:



IMPORTANT: The order we put the data into the database matters!

Add the following data to your tables:

address_type

address_type_id	type
1	work
2	home
3	public

address

address_id	house_number	street_name	city	province	country	frn_address_type_id
1	741	Heather St	Richmond	BC	Canada	2
2	10223	Shell St	Richmond	BC	Canada	2
3	#105 887	River Rd	Delta	BC	Canada	2
4	6088	Main St	Burnaby	BC	Canada	3

event

event_id	title	description	start_time	end_time	frn_host_contact_id	frn_address_id
1	Spring Break Kickoff	Pool party at the Tortia's	2018-03-15 18:00	2018-03-15 23:50	2	2

contact

contact_id	first_name	last_name	birth_date
1	Joe	Sorenson	1997-05-22 0:00
2	Sara	Tortia	1996-01-10 0:00
3	Lia	Tortia	1994-08-09 0:00
4	Michael	Park	1995-06-23 0:00

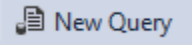
event_contact

event_contact_id	frn_event_id	frn_contact_id
1	1	1
2	1	2
3	1	3
4	1	4

contact_address

contact_address_id	frn_contact_id	frn_address_id
1	1	1
2	2	2
3	3	2
4	4	3

Click on New Query to open a new query window:

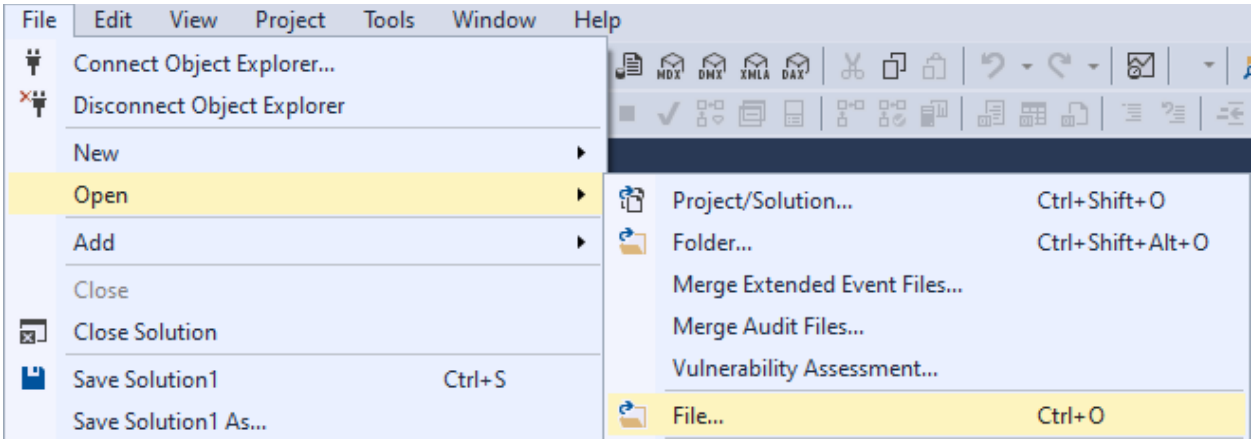


Run the following queries to test your tables and data:

1.
```
SELECT
      e.title,
      ea.house_number AS "event_house_number",
      ea.street_name AS "event_street_name",
      ea.city AS "event_city",
      h.first_name AS "host_name",
      i.first_name AS "guest_name"
FROM event AS e
INNER JOIN address AS ea
  ON e.frn_address_id = ea.address_id
INNER JOIN event_contact AS ec
  ON e.event_id = ec.frn_event_id
INNER JOIN contact AS i
  ON ec.frn_contact_id = i.contact_id
INNER JOIN contact AS h
  ON h.contact_id = e.frn_host_contact_id;
```

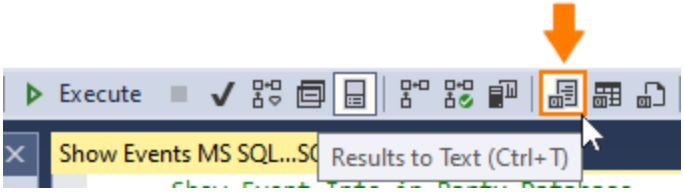
2.
```
SELECT
      c.first_name,
      c.last_name,
      a.house_number,
      a.street_name,
      a.city
FROM contact AS c
INNER JOIN contact_address AS ca
  ON c.contact_id = ca.frn_contact_id
INNER JOIN address AS a
  ON a.address_id = ca.frn_address_id;
```

Open the "Show Events MS SQL.sql" file in SSMS.

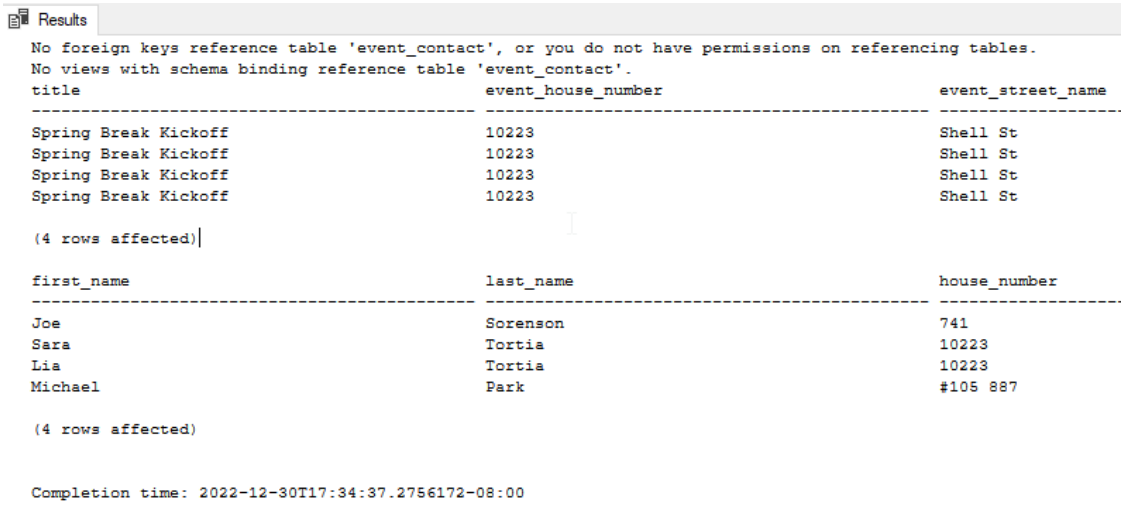


Normally when we run a query (or a set of queries) in SSMS, the results are displayed in a grid. As part of your submission for this lab, I would like you to submit the results as a text file.

To switch the results from grid to text file, look for the execute toolbar and switch the output to "Results to Text".

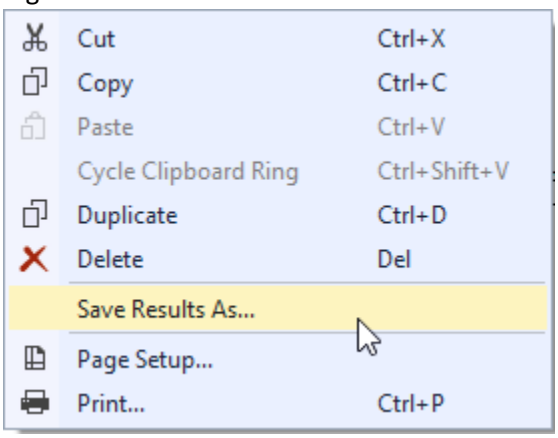


Click on Execute.
Your results should look like this (at the end of the results):



```
Results
No foreign keys reference table 'event_contact', or you do not have permissions on referencing tables.
No views with schema binding reference table 'event_contact'.
title                                          event_house_number                             event_street_name
---------------------------------------------- ---------------------------------------------- ------------------
Spring Break Kickoff                           10223                                          Shell St
Spring Break Kickoff                           10223                                          Shell St
Spring Break Kickoff                           10223                                          Shell St
Spring Break Kickoff                           10223                                          Shell St

(4 rows affected)

first_name                                     last_name                                      house_number
---------------------------------------------- ---------------------------------------------- ------------------
Joe                                            Sorenson                                       741
Sara                                           Tortia                                         10223
Lia                                            Tortia                                         10223
Michael                                        Park                                           #105 887

(4 rows affected)


Completion time: 2022-12-30T17:34:37.2756172-08:00
```

Right click on the results section and click "Save Results As…".



Save your results file as "Event_Results_MSSQL.rpt"

You will be marked on your accuracy and attention to detail for the table structure and data.

Make sure that:

- your tables are named correctly
- you have the correct columns with the correct data types
- the primary keys in all your tables are set to `Identity Column`
- that you have all the correct foreign key constraints on your table definitions
- all the correct people are invited to the correct parties at the correct addresses
- columns in the tables are marked as NOT NULL as described in the instructions

Submit your results file:
Your file should be around **70 Kbytes**.
`08_Event_Results_MSSQL.rpt`

The initial contents of `Event_Results_MSSQL.rpt` should look like this:

```
TABLE_CATALOG
-----------------
party
party
party
party
party
party

(6 rows affected)
```

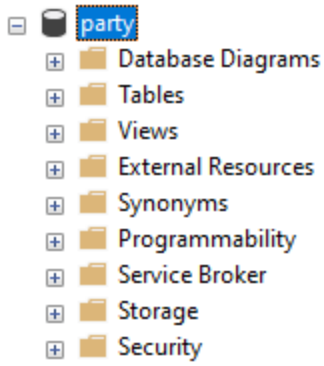
… and it should end with:

```
first_name                                         last_name
-------------------------------------------------- ----------------
Joe                                                Sorenson
Sara                                               Tortia
Lia                                                Tortia
Michael                                            Park

(4 rows affected)


Completion time: 2022-12-29T19:38:34.6427529-08:00
```

Step 6: Reverse Engineer our new database.

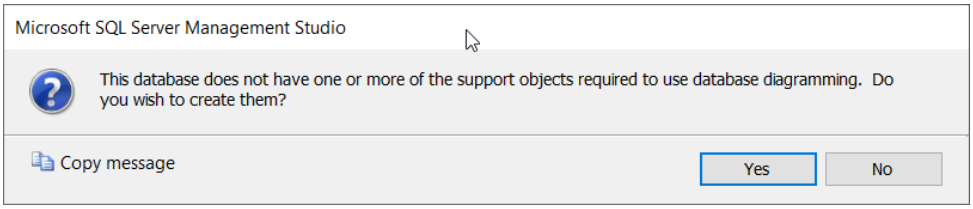
For existing databases, we can generate Entity Relational Diagrams (ERD) right from within the SSMS.

To start, make sure that the party database is expanded in the Object Explorer:



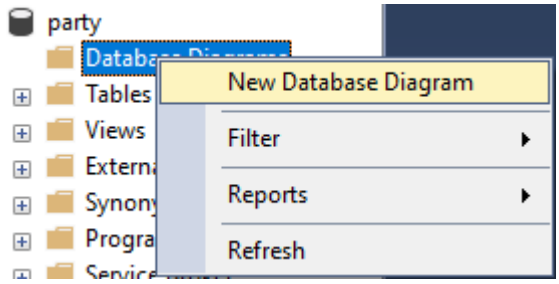
Click on "Database Diagrams" to expand it.

If you get a message like this:

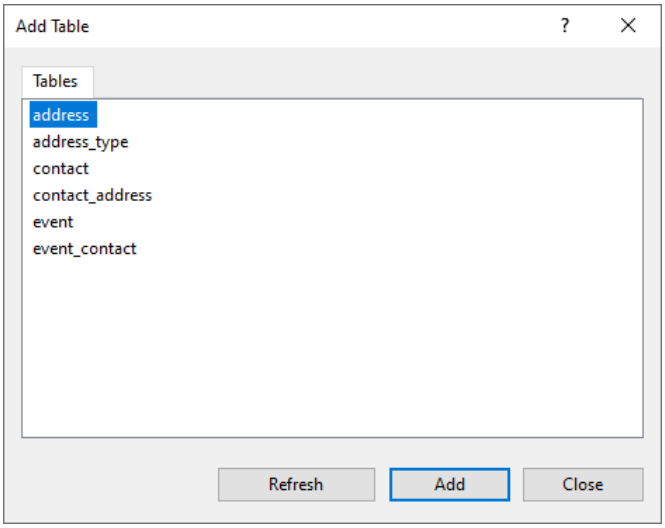


Click "Yes" to proceed.

Right click on the "Database Diagrams" and click "New Database Diagram".



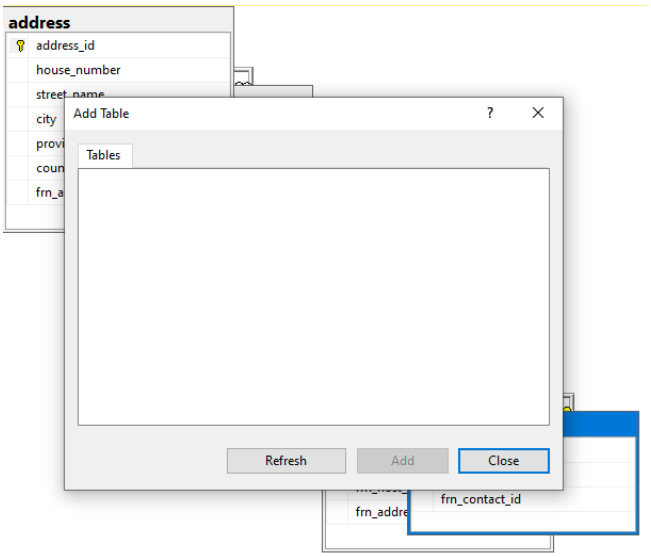
This popup will show all the tables in your database.



In this case we want to select all the tables so we can model our entire database.

It may be the case where you have a REALLY large database with lots of tables and you only want an ERD with a small subset of the tables.

Select your tables and click "Add". Clicking "Add" will add it to the ERD and remove it from the popup list. Continue adding until the list is empty and your ERD has all the tables.

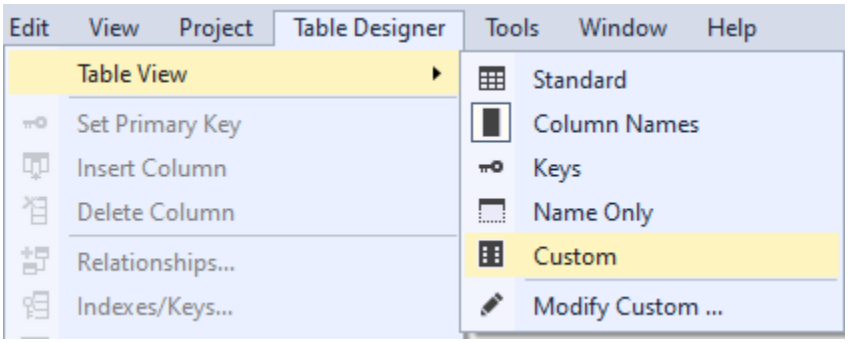


Click "Close".

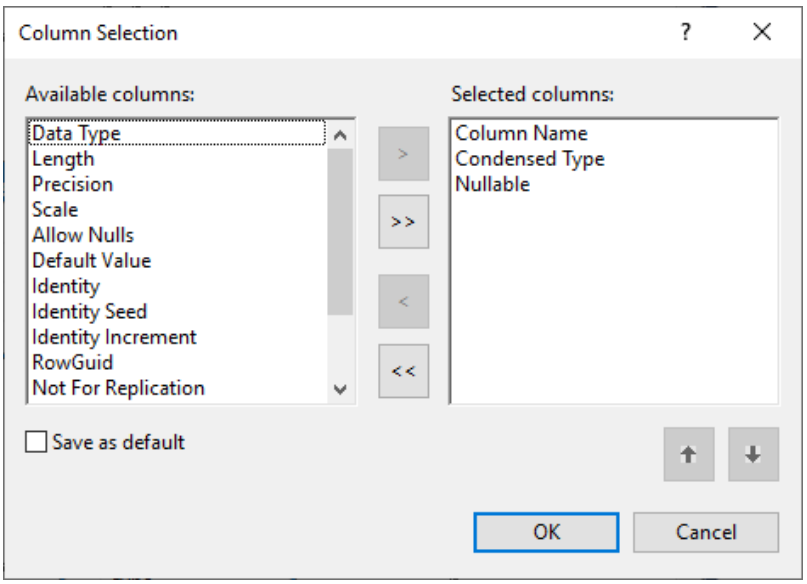
You should now see your ERD that SSMS has created for you.

Note SSMS will attempt to make the ERD look pretty by moving the tables so they and their relationships don't overlap. Have a look at the ERD and make sure you can see all tables and that no relationships cross each other or are obscured by any tables.

Let's make sure the data types of each column in the tables are visible. Do to this select all the tables and go to the "Table Designer" menu and clicking "Table View" and then "Custom".



The "Custom" view for a table on my SSMS shows the Column Name, Condensed Data Type and whether the column allows nulls or not:



If I set my contact table to

contact

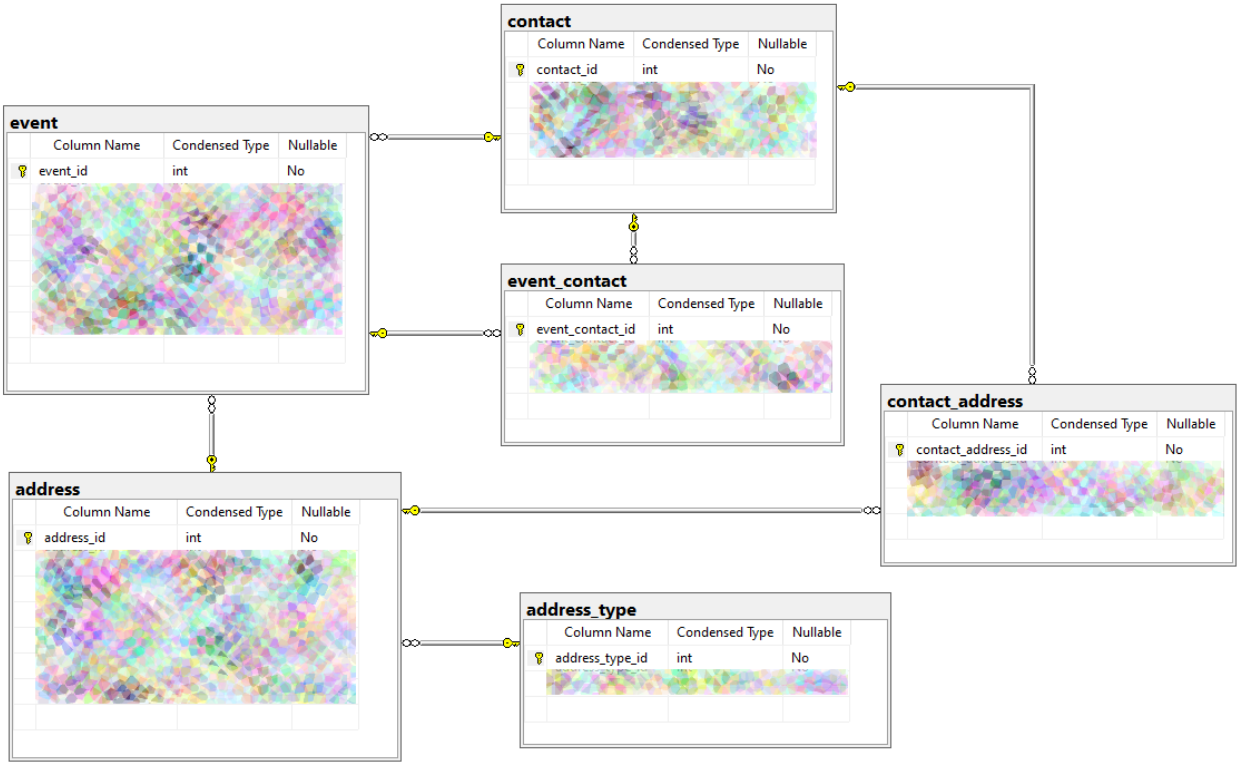
Column Name	Condensed Type	Nullable
contact_id	int	No
first_name	nvarchar(45)	No
last_name	nvarchar(45)	Yes
birth_date	datetime	Yes

Take a moment to rearrange the tables so that the foreign key lines are not crossing each other and to clean up the diagram.

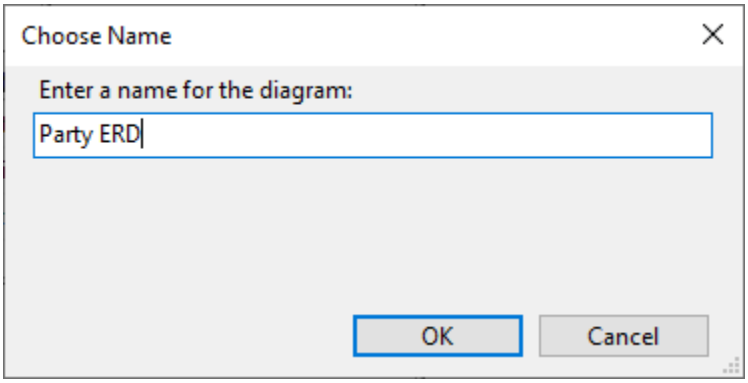
Make sure that none of the relationship lines are hidden behind other entities.
Also, you may need to expand the "Column Name" column to show the entire column name (sometimes the column names are too long for the default width and get cut off like `contact_address_id` in the example below).

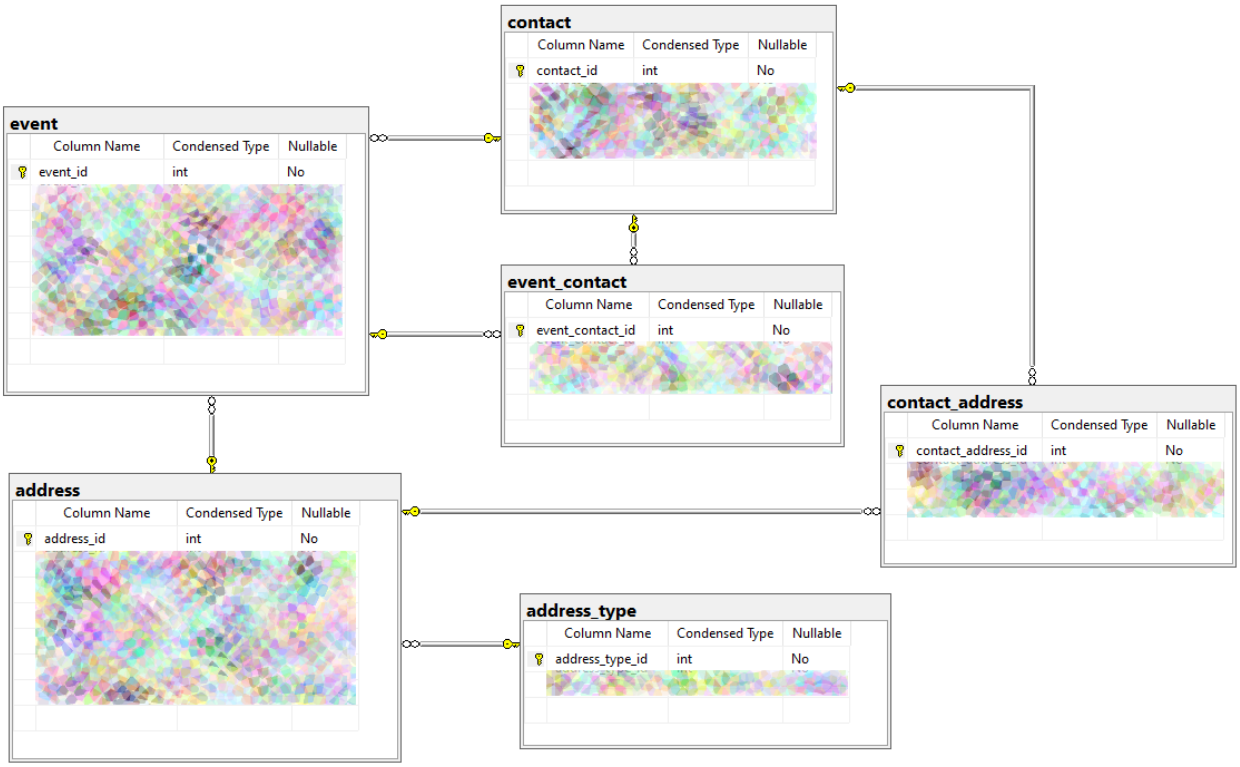
Column Name
contact_add...
frn_contact_id
frn_address_id

Your ERD should look something like this:



You can save the diagram by clicking the save icon and giving your diagram a name.



Submit your screenshot:
Filename: **09_MS_SQL_Reverse_Engineer.jpg**

Submit